NLP Assignment 3

Adam Fleisher Gal Barak Tamar Tabbach

Question 1: Prompting for Reasoning

1.2.d – Zero-Shot Accuracies and Model Comparison

Method	Accuracy
LLaMA zero-shot (temperature = 0)	0.68
GPT-2 zero-shot (temperature = 0)	0.48

LLaMA outperforms GPT-2 by 20 percentage points (0.68 vs. 0.48). Three differences that may explain this gap:

1. **Model scale and training.** LLaMA-3.1-8B is a modern instruction-tuned model trained on large-scale data for general reasoning and QA. GPT-2 (124M parameters for the hugging face version we are loading) is a much smaller, older autoregressive language model trained mainly on next-token prediction without instruction tuning.

2. **Prompting interface.** LLaMA is queried through a chat API with an explicit system prompt instructing the model to answer only `True` or `False`. GPT-2 has no system role; the instruction must be embedded in the raw text prompt, which is a weaker way to enforce output format, as we can see in the notebook a lot of these answers are neither correct or incorrect they are completely not in the format and therefore invalid (but marked incorrect).

3. **Task suitability.** StrategyQA requires multi-hop commonsense reasoning. LLaMA was trained on diverse instruction-following and reasoning data, while GPT-2 lacks this specialization and often generates free-form continuations rather than concise boolean answers.

1.3.b – Self-Consistency Accuracy (temperature = 1.0)

Using self-consistency with $n = 5$ samples and temperature = 1.0, the accuracy was: **0.64**

1.3.c – Temperature Analysis (0, 0.5, 1.0)

We repeated self-consistency ($n = 5$) at three temperature settings:

Temperature	Accuracy	Unanimous votes	Split votes
0.0	0.680	50/50 (100%)	0/50 (0%)
0.5	0.640	40/50 (80%)	10/50 (20%)
1.0	0.640	37/50 (74%)	13/50 (26%)

Vote distributions (selected patterns):

- **Temp = 0:** Only two patterns appeared: `True:0, False:5` (37 questions) and `True:5, False:0` (13 questions). Every question had identical votes across all 5 samples. This is because when temperature is set to 0, the model just takes argmax over its answer distribution making it deterministic, this means that self-consistency is meaningless when temperature is set to 0.
- **Temp = 0.5:** Here sampling adds randomness, but the distribution stays sharper than at temp 1.0, so high-probability answers are still chosen most of the time — hence mostly 5–0 votes, with occasional splits on uncertain questions.
- **Temp = 1.0:** Highest diversity: 13/50 questions had split votes.

Insights:

- **Temperature vs. diversity:** Lower temperature produces nearly deterministic outputs (100% unanimous at $T = 0$). Higher temperature increases output diversity and the frequency of split votes (26% at $T = 1.0$).
- **Self-consistency benefit:** In our case, since when $T = 0$ is equal to no majority voting at all, it seems that self consistency only damaged the results. This of-course does not prove anything since the experiment scope is relatively small and running the same experiments with a larger dataset or with different random seeds might provide different results.

1.4.d – ICL Accuracy ($n = 10$)

Using in-context learning with $n = 10$ demonstrations (10 random examples from `demos_dict`), temperature = 0: **0.62**

1.4.e — ICL Analysis

We analyzed how demonstration **quantity**, **ordering**, and **similarity** affect accuracy. All experiments used temperature 0 and the same evaluation pipeline as in 1.4.d. **Important Note:** Since the examples are chosen randomly, rerunning the experiment showed high variance in the results, we chose a representing experiment to base our conclusions on (averaging over many experiments takes considerable time with our computers).

Quantity of demonstrations.

Number of demos (n)	Accuracy
1	0.580
3	0.600
5	0.580
10	0.600
20	0.580

Accuracy was relatively unstable and did not increase monotonically with more demonstrations. The best result in this run was $n = 3$ and $n = 10$ (both 0.60), while $n = 1$, $n = 5$, and $n = 20$ were slightly lower (0.58). This suggests that more examples do not automatically help: very long prompts may dilute attention, and additional demos may add noise rather than useful signal. In our runs, performance stayed in a narrow band (0.58–0.60) for most values of n, with no clear benefit from scaling up the number of demonstrations.

Ordering of demonstrations ($n = 10$). To isolate ordering effects, we first sampled one fixed set of 10 demonstrations, then tested three orderings of the *same* demos:

Ordering	Accuracy
Original order	0.640
Reversed order	0.680
Shuffled order	0.540

Reversing the demonstrations improved accuracy slightly (0.68), while shuffling hurt (0.54). This indicates that order can matter, likely because recent/context-adjacent examples may influence the model more strongly. However, the effect is not fully consistent, so ordering alone does not explain ICL performance.

Similarity of demonstrations ($n = 10$). We compared: (i) 10 random demos per question, and (ii) 10 demos selected per question by highest word-overlap similarity (Jaccard similarity over question tokens).

Demo selection strategy	Accuracy
Random 10 demos	0.600
10 most similar demos per question	0.540

Similarity-based retrieval did *not* improve performance. Simple lexical overlap is a weak proxy for reasoning relevance on StrategyQA, so "similar-looking" questions may still teach the wrong reasoning pattern. This result initially surprised us but then we thought about it being some sort of overfitting, questions with overlapping word does not necessarily mean the same answer, especially in yes or no questions...

Summary. In our experiments, ICL gains were limited and sensitive to demo configuration. The strongest setting was reversed order with a fixed strong random subset (0.68), matching zero-shot, while random shuffling and similarity-based selection performed worse. Overall, ICL only helps when demonstrations are informative and well presented, otherwise it can underperform zero-shot.

1.5.c – Chain-of-Thought Accuracy

Using chain-of-thought prompting with a custom system prompt, temperature = 0, and `max_tokens = 1024`: **0.76**

CoT improved accuracy from 0.68 (zero-shot) to 0.76 (+8 points), showing that explicit step-by-step reasoning helps on multi-hop StrategyQA questions.

1.6.d – ICL + CoT Combined Accuracy

Using 1, 3, and 5 chain-of-thought demonstrations (with facts and decomposition) plus CoT prompting on the target question we got: **0.54, 0.60, 0.60** meaning 3 and 5 chain-of-thought demonstrations tied for the best results in this run.

1.6.e — Method Comparison

We compared all prompting methods on the 50-question StrategyQA test set using Llama 3.1-8B with temperature 0:

Method	Accuracy
Zero-shot	0.68
ICL ($n = 10$)	0.62
CoT	**0.76**
ICL + CoT ($n = 1$)	0.54
ICL + CoT ($n = 3$)	0.60
ICL + CoT ($n = 5$)	0.60

Discussion. Plain **ICL** was the weakest method among the main baselines (0.62), below zero-shot (0.68): providing only question–answer pairs, without any reasoning, gives the model a pattern to copy but no guidance on *how* to solve multi-hop questions.

Adding reasoning helps: **CoT** reached **0.76**, the best overall result, since letting the model think step by step before answering fits the multi-step nature of StrategyQA.

For **ICL + CoT** we varied the number of demonstrations with a fixed random demo set per setting. Performance was again sensitive to this choice, but it did *not* beat pure CoT: $n = 1$ reached only 0.54, while $n = 3$ and $n = 5$ both reached 0.60. Combining demonstrations with CoT therefore did not improve over CoT alone in this final run; the longer prompts likely added noise and made answer extraction harder, while a single demonstration was especially unhelpful.

Conclusion. Reasoning-based prompting clearly outperforms plain ICL, and **pure CoT (0.76)** was the strongest method overall. ICL + CoT remained unstable and underperformed CoT in our final results, suggesting that adding solved examples is not automatically beneficial once the model is already prompted to reason step by step. The method also appears sensitive to the chosen demonstrations, prompt length, and API runtime conditions, so a more comprehensive experiment averaged over multiple demo sets would be needed to draw stronger conclusions.

Question 2: WhoDunnit (Multi-Agent Systems)

2.1 Naive Agent Implementation (Analysis)

We ran the naive baseline with the default configuration and no additional flags (`python run_game.py --iterations 3`: 3 suspects, `max_turn_count = 11`, no summarizer, no internal thought). *Alex* is the Accuser and *Beth* the Intel agent.

What specific behavioral problems did you identify in the initial run of the naive agents?

Problem A — Analysis paralysis: the Accuser will not accuse even when a single answer already fixes the culprit. In **Game 1** (culprit = Suspect 3) only Suspect 3 wears square glasses, so Beth's answer `square eye glasses` → `Suspect 3` uniquely determined the culprit on the second question. Alex nonetheless asked four more broad questions and never accused, hitting the turn limit.

Problem B — No cross-turn reasoning: the Accuser never intersects the narrowing suspect lists into a running candidate set. In **Game 2** (culprit = Suspect 3) no single answer pins the culprit; it emerges only from combining `blue eye color` → `S2, S3` with `circular eye glasses` → `S1, S3`, whose intersection is {S3}. Because Alex treats each answer in isolation, it never computes this intersection, keeps questioning, and times out. This inability to accumulate evidence across turns is the underlying cause of the analysis paralysis in Problem A.

Do the agents dynamically utilize the full spectrum of their available action space, or do they prematurely converge on a single, repetitive action type?

They **converge**. Alex issues only `request-broad` in all three games (zero `request-specific`; `accuse` only once, in Game 3), which in turn forces Beth into `respond-broad` only. Because the terminal `accuse` action is under-used, information accumulates but is rarely acted upon, so games stall: Games 1 and 2 exhaust all 11 turns without an accusation, ending in timeout rather than a decision.

Does the Accuser make a blind accusation prematurely, or does it get stuck in an endless loop until the `max-turn-count` is reached? Why do you think it struggles to finalize the decision?

It gets stuck in an **endless questioning loop**, not a premature accusation: Games 1 and 2 reach the `max-turn-count` with no accusation, and Game 3 accuses correctly only after redundant confirmation. It struggles to finalize because:

1. **Stateless prompting** — no maintained candidate set, so it never sees that one suspect remains;
2. the naive prompt **forbids reasoning**, so the model defaults to asking one more question rather than committing;
3. **no firm stopping criterion**, so "accuse when certain" never triggers.

2.2 Context Management via Summarization

We implemented `summarize` in `SummerizerAgent` as an evidence-preserving LLM rewrite of the channel (`temperature`=0.2, prompt instructed to keep every per-suspect confirmed/excluded attribute and never invent facts) and ran with the summarizer enabled. The Summarizer is *Charlie*.

Did the summarizer inadvertently remove any critical clues that led to the Accuser failing to identify the culprit?

No — no clue was dropped. Every decisive fact survived, including the culprit-identifying one (only Suspect 3 has blue eyes). The failure came from two other distortions. First, Charlie downgraded *certainty*: Beth's definite `circular eye glasses` → `Suspect 1, Suspect 2, Suspect 3` became "Suspect 1: *possibly* has circular eye glasses" (likewise S2, S3), turning a confirmed fact ambiguous. Second, the per-suspect summary replaced the verbatim Q&A record, so Alex lost track of which questions it had already asked. Both pushed Alex to re-ask "circular eye glasses" and "blue eye color" until it timed out — despite the culprit clue being present the whole time.

Did the reduction in context length result in more coherent questions responses from the Accuser and Intel agents, or did the loss of "raw" dialogue history make their behavior less natural?

Mixed. The questioning became **more coherent** in 2 of 3 games: the per-suspect summaries (e.g. Game 2: "Suspect 2: blue eye color, wears square eye glasses, has long hair") kept Alex focused on one suspect and pushed it toward targeted `request-specific` questions, unlike the naive baseline's endless broad queries. But coherence did not translate into success — **all three summarizer games still timed out** (0/3, versus 1/3 for the naive baseline). Replacing the raw dialogue with a summary removed Alex's memory of what it had already asked, so it kept reconfirming known traits (Game 1's vague "possibly has" made this worse). So compression made the *style* of questioning cleaner but the *behavior* less effective: the raw history, for all its bulk, was what let Alex avoid repeating itself.

Based on your observations, do you believe this summarization approach would scale to a game with a much larger suspect pool (e.g., `suspect-count=20`)?

Not without changes. (1) A free-text LLM rewrite is lossy: the "possibly" distortion seen at 3 suspects will compound at 20, and a single weakened or dropped fact can leave the culprit ambiguous. (2) The summary must record facts for many suspects, so its length grows with the pool and the "concise channel" benefit erodes. (3) The real bottleneck is the Accuser's reasoning and stopping ability, which summarization does not address — in fact it worsened matters at N=3 (0/3 solved vs. 1/3 naive), since a third of all turns are consumed by summarizing. A deterministic, structured state (a maintained per-suspect table of confirmed/excluded attributes) would scale far better than a natural-language summary.

2.3 Implementing Reasoning Agents

We ran with the `--allow-internal-thought` flag and implemented `take_turn_thought` for both the Intel and the Accuser. Each agent may now reason freely, but still emits the environment's required final `ACTION/INFO` block.

Detail your architectural decisions regarding prompt engineering. What specific intermediate reasoning steps did you introduce to guide the agents, and what did you want each agent to analyze internally before generating its environment-compliant response?

Each agent reasons freely in the prompt but must end with the environment's `ACTION/INFO` block; we read only the *last* such block, and for an accusation we extract the suspect number and reformat it to the exact `"Suspect N"` string the environment requires. The reasoning steps we asked each agent to perform internally were:

- **Accuser** — (1) extract the culprit's attributes from its description; (2) for every suspect, mark which attributes are confirmed/excluded and maintain the set of suspects consistent with *all* evidence so far; (3) accuse iff exactly one candidate remains, otherwise ask the single most-informative, non-repeated question. The intended artifact is the *running candidate intersection* — exactly what the naive agent never computed — and we pass `turns_left` so it trades certainty against the clock.
- **Intel** — (1) classify the question as *specific* (Yes/No) or *broad*; (2) identify the relevant attribute; (3) check each suspect against it, so broad lists are complete and Yes/No answers correct.

Evaluate whether this "Chain-of-Thought" mechanism successfully mitigated the Task 1 failure modes. Did adding this internal reasoning step improve agent coordination, action-space utilization, or overall success rate compared to the naive baseline?

Yes, clearly, on all three axes:

- **Coordination:** Intel's lists are accurate and the Accuser *consumes* them by intersecting — e.g. Game 1 (`black hat`→{S3}, `square glasses`→{S3}) and Game 3 (`green eyes`→{S1,S2} ∩ `red shoes`→{S1,S3} = {S1}).
- **Action-space utilization:** the Accuser now actually reaches `accuse` and mixes broad with specific questions; the terminal action is no longer neglected. Intel likewise exercised both actions: `respond-broad` throughout the 3-suspect runs and `respond` (Yes/No) in the scaling runs (e.g. `Does Suspect 3 have blue eyes?`→ *Yes* at N=5, and `Does Suspect 8 have a bronze watch?`→ *Yes* at N=16).
- **Success rate:** 2/3 solved in the 3-suspect batch, versus 1/3 for the naive baseline. The endless-questioning loop is largely fixed.

If the agents still fail, discuss the underlying causes. If successful, scale the suspect count and determine the maximum your agents can process (subject to the turn limit and API rate restrictions); analyze the factors that contribute to failure at scale.

Residual failure. In one 3-suspect game (Game 2) the intersection was already {S3} after just *two* broad answers (`black hat`→{S1,S3} ∩ `brown eyes`→{S2,S3} = {S3}), yet the Accuser asked three more broad questions and then a redundant specific question, and timed out. This is a symptom of *over-verification* (see below): the model reaches the answer but keeps asking, and with only ≈6 Accuser moves at the 11-turn limit, one or two wasted questions cause a timeout.

Scaling. We increased the suspect count with the turn limit set to 3× suspects, and the agents solved every configuration correctly from 5 up to **32 suspects**, always identifying and accusing the

culprit. N=32 is the largest we could test: an N=64 game needs $\approx$ 192 turns, each re-embedding all 64 descriptions, which exceeds the Groq free-tier daily token cap (`Limit 100000`) outright, so those attempts never completed. The cap therefore bounds *throughput*, not the agents' reasoning, and N=32 stands as the demonstrated maximum under the free tier.

Factors that limit scaling. (1) **Over-verification** is the dominant inefficiency: the Accuser consistently asks several more questions than needed to reach a singleton (e.g. 5 needed vs. 11 asked at N=32). This is harmless when turns are plentiful but is precisely what causes timeouts under a tight limit — so the required turn budget, not correctness, is the practical ceiling. (2) **API budget:** each 70B turn ($\approx$1024-token thoughts plus an Intel prompt that grows with N) is token-expensive, so throughput — how many and how large the games can be per day — is bounded by the free-tier daily cap even when a single game's reasoning would succeed. (3) **Intel accuracy at scale:** with 20+ descriptions to scan per broad question there is more room for a missed or misclassified suspect, which would corrupt the intersection.

Propose three modifications that could further enhance agent performance and improve the overall success rate of the system.

1. **Deterministic candidate tracking in code.** Maintain a per-suspect table of confirmed/excluded attributes parsed from the Q&A and auto-accuse once a single candidate remains; let the LLM only *choose questions*. This removes reliance on the model's mental set-intersection and directly eliminates the over-verification that causes timeouts.
2. **Information-maximizing questions.** Instruct the Accuser to pick the attribute whose answer most evenly splits the remaining candidates (binary-search/entropy style) — fewer turns needed, so larger pools fit under the turn limit.
3. **Compound (multi-attribute) questions.** Extend the action space so the Accuser can ask for several attributes at once (e.g. "which suspects wear glasses *and* have blue eyes?"), letting the Intel agent return the intersection directly. This eliminates more candidates per turn and cuts the number of questions needed to reach a singleton.

Question 3: Textual Search

1. **Search results.**

   Sparse model results

   What is the capital of New Zealand?

   - New Zealand — 0.9083811995622542
   - New Zealand's capital city is Wellington, and its most populous city is Auckland — 0.7285520507599198
   - The South Island is the largest landmass of New Zealand — 0.4824231442212654

   What currency is used in New Zealand?

   - New Zealand — 0.9083811995622542
   - The word today is increasingly used to refer to all non-Polynesian New Zealanders — 0.46642594407468546
   - While the demonym for a New Zealand citizen is New Zealander, the informal "Kiwi" is commonly used both internationally and by locals — 0.4540587660165493

   Should I visit New Zealand if I am interested in sightseeing?

   - New Zealand — 0.9083811995622542
   - If no majority is formed, a minority government can be formed if support from other parties during confidence and supply votes is assured — 0.443289725141427O6
   - The most prominent differences between the New Zealand English dialect and other English dialects are the shifts in the short front vowels: the short-i sound (as in kit) has centralised towards the schwa sound (the a in comma and about); the short-e sound (as in dress) has moved towards the short-i sound; and the short-a sound (as in trap) has moved to the short-e sound — 0.39941400980567277

   Dense model results

   What is the capital of New Zealand?

   - New Zealand's capital city is Wellington, and its most populous city is Auckland — 0.77611506
   - New Zealand () is an island country in the southwestern Pacific Ocean — 0.65163213
   - New Zealand country profile from BBC News
     Te Ara: The Encyclopedia of New Zealand
     New Zealand — 0.6514702

   What currency is used in New Zealand?

   - The currency is the New Zealand dollar, informally known as the "Kiwi dollar"; it also circulates in the Cook Islands (see Cook Islands dollar), Niue, Tokelau, and the Pitcairn Islands — 0.76597565
   - New Zealand's main trading partners, , are China (NZ$27 — 0.59499335

- 6%) to New Zealand's total GDP and supporting 7 — 0.57041574

Should I visit New Zealand if I am interested in sightseeing?

- New Zealand is known for its extreme sports, adventure tourism and strong mountaineering tradition, as seen in the success of notable New Zealander Sir Edmund Hillary — 0.53034496
- New Zealand country profile from BBC News
  Te Ara: The Encyclopedia of New Zealand
  New Zealand — 0.51355964
- New Zealand art and craft has gradually achieved an international audience, with exhibitions in the Venice Biennale in 2001 and the "Paradise Now" exhibition in New York in 2004 — 0.5126652

2. Sparse (TF-IDF) search is better when relevance depends on exact keyword overlap, such as rare terms, proper nouns, or technical codes that must match literally. Dense search is better when relevance is semantic, since it matches meaning and can retrieve passages that use synonyms or paraphrases the query never mentions.

Question 4: AI Disclosure & Reflection

1. Did you use AI? Yes. We used Anthropic's **Claude** (via the Claude Code CLI, Opus-class models) and, for occasional quick lookups, **ChatGPT**. The models under study in the assignment itself (`llama-3.1-8b-instant`, `llama-3.3-70b-versatile` via the Groq API, GPT-2, and `all-mpnet-base-v2`) were used as the objects of the experiments rather than as authoring aids.

2. How and why did you use it? We used AI as a coding and writing assistant, not as a substitute for our own understanding. Concretely: (i) **code generation and debugging** — scaffolding helper functions (e.g. answer normalization, the chain-of-thought answer extractor, and parsing the agents' final `ACTION/INFO` block), and diagnosing errors during the WhoDunnit runs; (ii) **editing and structuring the written report** — tightening phrasing and organizing the analysis. All experimental results, accuracy numbers, and game logs reported here were produced by running our own code, and we verified every claim against the actual notebook outputs and logs before including it.

3. Reflection. AI assistance was most helpful for boilerplate and for quickly surfacing bugs, which let us spend more time on the conceptual parts — designing prompts and interpreting the results. What hindered us more than the tooling were the Groq free-tier rate and token limits, which constrained how many large-scale WhoDunnit games we could run. Our main takeaway is that AI is a strong accelerator for implementation and drafting, but the analysis and validation still require careful human judgment: the model can propose a plausible explanation, yet it is our responsibility to check it against the evidence.